Phase 2 — Multi-Provider Streaming Search Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Wire multi-provider streaming search from Android → Go API SSE → archive.org + gutenberg scrapers, add provider labels and multi-select filter to search UI.

Architecture: New `/v1/search` SSE endpoint in Go API fans out to existing provider scrapers. Android `SseClient` consumes the stream. `SearchResultViewModel` renders results incrementally with per-item provider chips. `ProviderChipBar` below search input lets users pick which providers to query.

Tech Stack: Go/Gin SSE (gin.Context.Stream), Kotlin Coroutines Flow, OkHttp streaming response, Jetpack Compose Material 3

Key Finding: Go Scrapers Already Implemented

`internal/archiveorg/client.go` and `internal/gutenberg/client.go` are fully functional — `Client.Search()` makes real HTTP calls to archive.org and gutendex.com. Both pass their unit tests. The missing piece is the multi-provider SSE handler that fans out to them and the Android client that consumes it.

Task 1: Add Multi-Search SSE Handler to Go API

Files:

- Modify: `lava-api-go/internal/handlers/v1/search.go`
- Modify: `lava-api-go/internal/handlers/v1/handlers.go`

Step 1: Add SSE event types and multi-search handler struct

Append to `lava-api-go/internal/handlers/v1/search.go`:

```
import (
    "encoding/json"
    "fmt"
    "io"
    "net/http"
    "sync"
    "time"

    "github.com/gin-gonic/gin"

    "digital.vasic.lava.apigo/internal/provider"
)

// sseEvent represents a single SSE event pushed to the client.
type sseEvent struct {
    Event string
    Data  string
}

// streamEvent writes a single SSE event to the client.
func streamEvent(w io.Writer, evt sseEvent) error {
    if evt.Event != "" {
        if _, err := fmt.Fprintf(w, "event: %s\n", evt.Event);
        err != nil {
            return err
        }
    }
    if _, err := fmt.Fprintf(w, "data: %s\n\n", evt.Data); err !=
        nil {
        return err
    }
    if f, ok := w.(http.Flusher); ok {
        f.Flush()
    }
    return nil
}

// MultiSearchHandler fans out search to multiple providers and
        streams
// results via SSE. It is registered on a standalone route (/v1/
        search)
// separate from the per-provider /v1/:provider/search routes.
type MultiSearchHandler struct {
    registry *provider.ProviderRegistry
}

// NewMultiSearchHandler creates a MultiSearchHandler with the
        given provider registry.
```

```
func NewMultiSearchHandler(reg *provider.ProviderRegistry)
        *MultiSearchHandler {
    return &MultiSearchHandler{registry: reg}
}

// providerStreamStatus tracks progress for one provider during
        streaming.
type providerStreamStatus struct {
    ProviderID   string `json:"provider_id"`
    DisplayName  string `json:"display_name"`
    ResultCount  int    `json:"result_count"`
    Page         int    `json:"page"`
    TotalPages   int    `json:"total_pages"`
    Error        string `json:"error,omitempty"`
}

// GetMultiSearch handles GET /v1/search?
        q=...&providers=id1,id2,...
func (h *MultiSearchHandler) GetMultiSearch(c *gin.Context) {
    query := c.Query("q")
    if query == "" {
        c.JSON(http.StatusBadRequest, gin.H{"error": "query
        parameter 'q' is required"})
        return
    }

    // Resolve providers — default to all SEARCH-capable if not
        specified.
    providerIDs := parseProviderList(c.Query("providers"))
    if len(providerIDs) == 0 {
        for _, id := range h.registry.IDs() {
            if h.registry.Supports(id, provider.CapSearch) {
                providerIDs = append(providerIDs, id)
            }
        }
    }
    if len(providerIDs) == 0 {
        c.JSON(http.StatusBadRequest, gin.H{"error": "no search-
        capable providers available"})
        return
    }

    // Build SearchOpts from query params.
    opts := provider.SearchOpts{Query: query}
    if v := c.Query("sort"); v != "" {
        opts.Sort = v
    }
    if v := c.Query("order"); v != "" {
        opts.Order = v
    }
    if v := c.Query("category"); v != "" {
        opts.Category = v
    }
```

```
c.Header("Content-Type", "text/event-stream")
c.Header("Cache-Control", "no-cache")
c.Header("Connection", "keep-alive")
c.Header("X-Accel-Buffering", "no")

totalProviders := len(providerIDs)
var mu sync.Mutex
searched := 0
failed := 0
totalResults := 0

c.Stream(func(w io.Writer) bool {
    for _, pid := range providerIDs {
        p := h.registry.Get(pid)
        if p == nil {
            continue
        }

        // Emit provider_start.
        startEvt := providerStreamStatus{
            ProviderID:  pid,
            DisplayName: p.DisplayName(),
        }
        data, _ := json.Marshal(startEvt)
        _ = streamEvent(w, sseEvent{Event: "provider_start",
    Data: string(data)})

        // Search with short timeout.
        ctx, cancel :=
    context.WithTimeout(c.Request.Context(), 30*time.Second)
        result, err := p.Search(ctx, opts,
    provider.Credentials{Type: "none"})
        cancel()

        if err != nil {
            mu.Lock()
            failed++
            mu.Unlock()
            errEvt := providerStreamStatus{
                ProviderID:  pid,
                DisplayName: p.DisplayName(),
                Error:       err.Error(),
            }
            data, _ := json.Marshal(errEvt)
            _ = streamEvent(w, sseEvent{Event:
    "provider_error", Data: string(data)})
            continue
        }

        // Stream results page.
        pageData := map[string]interface{}{
            "provider_id":  pid,
            "display_name": p.DisplayName(),
```

```
                "items":        result.Results,
                "page":         result.Page,
                "total_pages":  result.TotalPages,
            }
            pageJSON, _ := json.Marshal(pageData)
            _ = streamEvent(w, sseEvent{Event: "results", Data:
        string(pageJSON)})

            mu.Lock()
            searched++
            totalResults += len(result.Results)
            mu.Unlock()

            doneEvt := providerStreamStatus{
                ProviderID:  pid,
                DisplayName: p.DisplayName(),
                ResultCount: len(result.Results),
                Page:        result.Page,
                TotalPages:  result.TotalPages,
            }
            data, _ = json.Marshal(doneEvt)
            _ = streamEvent(w, sseEvent{Event: "provider_done",
        Data: string(data)})
        }

        // Emit stream_end.
        endData := map[string]interface{}{
            "providers_searched": searched,
            "providers_failed":   failed,
            "total_results":      totalResults,
            "total_providers":    totalProviders,
        }
        endJSON, _ := json.Marshal(endData)
        _ = streamEvent(w, sseEvent{Event: "stream_end", Data:
        string(endJSON)})
        return false
    })
}

// parseProviderList splits a comma-separated provider list,
        trimming whitespace
// and filtering empty entries.
func parseProviderList(raw string) []string {
    if raw == "" {
        return nil
    }
    parts := strings.Split(raw, ",")
    result := make([]string, 0, len(parts))
    for _, p := range parts {
        p = strings.TrimSpace(p)
        if p != "" {
            result = append(result, p)
        }
```

```
    }
    return result
}
```

Note: Add "context" and "strings" and "sync" to the imports in search.go.

- [ ] **Step 2: Register the multi-search route in main.go**

In lava-api-go/cmd/lava-api-go/main.go, in buildRouter(), after the v1 group registration:

```
// v1 multi-provider search (SSE streaming — resolves providers
        dynamically)
router.GET("/v1/search",
        v1handlers.NewMultiSearchHandler(deps.Registry).GetMultiSearch)
```

Place this between the v1 group registration and return router:

```
    // v1 provider-agnostic routes
    v1 := router.Group("/v1/:provider")
    v1handlers.Register(v1, &v1handlers.Deps{Cache: deps.Cache})

    // v1 multi-provider SSE search
    router.GET("/v1/search",
        v1handlers.NewMultiSearchHandler(deps.Registry).GetMultiSearch)

    return router
```

- [ ] **Step 3: Run Go build to verify it compiles**

Run: cd lava-api-go && go build ./... Expected: BUILD SUCCESSFUL

- [ ] **Step 4: Commit**

```
git add lava-api-go/internal/handlers/v1/search.go lava-api-go/
        internal/handlers/v1/handlers.go lava-api-go/cmd/lava-api-
        go/main.go
git commit -m "feat(api): multi-provider SSE search handler

Adds GET /v1/search SSE endpoint that fans out to all registered
providers and streams results as they arrive. Each provider gets a
30s timeout. Events: provider_start, results, provider_done,
provider_error, stream_end.

Bluff-Audit: N/A (this is the implementation; tests follow)
Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>"
```

Task 2: Go API Handler Tests

Files:

- Modify: `lava-api-go/internal/handlers/v1/search_test.go`

☐ **Step 1: Write failing test for multi-search handler**

```
package v1

import (
    "bufio"
    "encoding/json"
    "net/http"
    "net/http/httptest"
    "strings"
    "testing"

    "github.com/gin-gonic/gin"

    "digital.vasic.lava.apigo/internal/provider"
)

func TestMultiSearchHandler_StreamsResultsForRegisteredProviders(t
        *testing.T) {
    gin.SetMode(gin.TestMode)

    reg := provider.NewRegistry()
    reg.Register(&fakeProvider{id: "test1", name: "Test One",
        searchResult: &provider.SearchResult{
        Provider:   "test1",
        Page:       1,
        TotalPages: 1,
        Results:    []provider.SearchItem{{ID: "1", Title: "Item
        One"}},
    }})
    reg.Register(&fakeProvider{id: "test2", name: "Test Two",
        searchResult: &provider.SearchResult{
        Provider:   "test2",
        Page:       1,
        TotalPages: 1,
        Results:    []provider.SearchItem{{ID: "2", Title: "Item
        Two"}},
    }})

    handler := NewMultiSearchHandler(reg)
    router := gin.New()
    router.GET("/v1/search", handler.GetMultiSearch)

    req := httptest.NewRequest(http.MethodGet, "/v1/search?
        q=test&providers=test1,test2", nil)
    w := httptest.NewRecorder()
```

```
router.ServeHTTP(w, req)

resp := w.Result()
if resp.StatusCode != http.StatusOK {
    t.Fatalf("expected 200, got %d", resp.StatusCode)
}
ct := resp.Header.Get("Content-Type")
if !strings.Contains(ct, "text/event-stream") {
    t.Fatalf("expected text/event-stream, got %s", ct)
}

// Parse SSE events from body.
scanner := bufio.NewScanner(resp.Body)
var events []map[string]string
current := make(map[string]string)
for scanner.Scan() {
    line := scanner.Text()
    if line == "" {
        if len(current) > 0 {
            events = append(events, current)
            current = make(map[string]string)
        }
        continue
    }
    if strings.HasPrefix(line, "event: ") {
        current["event"] = strings.TrimPrefix(line, "event: ")
    }
    if strings.HasPrefix(line, "data: ") {
        current["data"] = strings.TrimPrefix(line, "data: ")
    }
}
if len(current) > 0 {
    events = append(events, current)
}

// Verify event sequence: provider_start × 2, results × 2,
    provider_done × 2, stream_end
if len(events) < 7 {
    t.Fatalf("expected at least 7 events, got %d: %v",
    len(events), events)
}

eventTypes := make([]string, len(events))
for i, e := range events {
    eventTypes[i] = e["event"]
}

assertEventPresent(t, eventTypes, "provider_start")
assertEventPresent(t, eventTypes, "results")
assertEventPresent(t, eventTypes, "provider_done")
assertEventPresent(t, eventTypes, "stream_end")

// Verify stream_end contains correct counts.
```

```
    for _, e := range events {
        if e["event"] == "stream_end" {
            var end map[string]interface{}
            json.Unmarshal([]byte(e["data"]), &end)
            if end["providers_searched"] != float64(2) {
                t.Errorf("expected 2 searched, got %v",
        end["providers_searched"])
            }
            if end["total_results"] != float64(2) {
                t.Errorf("expected 2 total_results, got %v",
        end["total_results"])
            }
        }
    }
}

func TestMultiSearchHandler_MissingQuery(t *testing.T) {
    gin.SetMode(gin.TestMode)

    reg := provider.NewRegistry()
    handler := NewMultiSearchHandler(reg)
    router := gin.New()
    router.GET("/v1/search", handler.GetMultiSearch)

    req := httptest.NewRequest(http.MethodGet, "/v1/search", nil)
    w := httptest.NewRecorder()
    router.ServeHTTP(w, req)

    if w.Code != http.StatusBadRequest {
        t.Errorf("expected 400, got %d", w.Code)
    }
}

func TestMultiSearchHandler_DefaultsToAllWhenNoProvidersParam(t
        *testing.T) {
    gin.SetMode(gin.TestMode)

    reg := provider.NewRegistry()
    reg.Register(&fakeProvider{id: "auto1", name: "Auto One",
        searchResult: &provider.SearchResult{
        Provider:   "auto1",
        Page:       1,
        TotalPages: 1,
        Results:    []provider.SearchItem{{ID: "a1", Title: "Auto
        Item"}},
    }})

    handler := NewMultiSearchHandler(reg)
    router := gin.New()
    router.GET("/v1/search", handler.GetMultiSearch)

    req := httptest.NewRequest(http.MethodGet, "/v1/search?
        q=test", nil)
    w := httptest.NewRecorder()
```

```
    router.ServeHTTP(w, req)

    if w.Code != http.StatusOK {
        t.Fatalf("expected 200, got %d", w.Code)
    }
    body := w.Body.String()
    if !strings.Contains(body, "provider_start") {
        t.Error("expected provider_start in response")
    }
}

func TestMultiSearchHandler_ProviderErrorEmitted(t *testing.T) {
    gin.SetMode(gin.TestMode)

    reg := provider.NewRegistry()
    reg.Register(&fakeProvider{id: "failing", name: "Failing",
        searchErr: fmt.Errorf("upstream down")})

    handler := NewMultiSearchHandler(reg)
    router := gin.New()
    router.GET("/v1/search", handler.GetMultiSearch)

    req := httptest.NewRequest(http.MethodGet, "/v1/search?
        q=test&providers=failing", nil)
    w := httptest.NewRecorder()
    router.ServeHTTP(w, req)

    body := w.Body.String()
    if !strings.Contains(body, "provider_error") {
        t.Error("expected provider_error in response")
    }
    if !strings.Contains(body, "upstream down") {
        t.Error("expected error message in response")
    }
}

func assertEventPresent(t *testing.T, events []string, eventType
        string) {
    t.Helper()
    for _, e := range events {
        if e == eventType {
            return
        }
    }
    t.Errorf("expected event type %q not found in %v", eventType,
        events)
}

// fakeProvider implements provider.Provider for testing.
type fakeProvider struct {
    id           string
    name         string
    searchResult *provider.SearchResult
```

```
    searchErr    error
}

func (f *fakeProvider) ID()
        string                                  { return f.id }
func (f *fakeProvider) DisplayName()
        string                             { return f.name }
func (f *fakeProvider) Capabilities()
        []provider.ProviderCapability { return
        []provider.ProviderCapability{provider.CapSearch} }
func (f *fakeProvider) AuthType()
        provider.AuthType                  { return
        provider.AuthNone }
func (f *fakeProvider) Encoding()
        string                             { return "UTF-8" }
func (f *fakeProvider) Search(ctx context.Context, opts
        provider.SearchOpts, cred provider.Credentials)
        (*provider.SearchResult, error) {
    if f.searchErr != nil {
        return nil, f.searchErr
    }
    return f.searchResult, nil
}
func (f *fakeProvider) Browse(ctx context.Context, categoryID
        string, page int, cred provider.Credentials)
        (*provider.BrowseResult, error) { return nil,
        provider.ErrUnsupported }
func (f *fakeProvider) GetForumTree(ctx context.Context, cred
        provider.Credentials) (*provider.ForumTree,
        error)                             { return nil,
        provider.ErrUnsupported }
func (f *fakeProvider) GetTopic(ctx context.Context, id string,
        page int, cred provider.Credentials)
        (*provider.TopicResult, error)        { return nil,
        provider.ErrUnsupported }
func (f *fakeProvider) GetTorrent(ctx context.Context, id string,
        cred provider.Credentials) (*provider.TorrentResult,
        error)              { return nil,
        provider.ErrUnsupported }
func (f *fakeProvider) DownloadFile(ctx context.Context, id
        string, cred provider.Credentials)
        (*provider.FileDownload, error)            { return nil,
        provider.ErrUnsupported }
func (f *fakeProvider) GetComments(ctx context.Context, id
        string, page int, cred provider.Credentials)
        (*provider.CommentsResult, error)  { return nil,
        provider.ErrUnsupported }
func (f *fakeProvider) AddComment(ctx context.Context, id,
        message string, cred provider.Credentials) (bool,
        error)                          { return false,
        provider.ErrUnsupported }
func (f *fakeProvider) GetFavorites(ctx context.Context, cred
        provider.Credentials) (*provider.FavoritesResult,
        error)                     { return nil,
        provider.ErrUnsupported }
```

```
func (f *fakeProvider) AddFavorite(ctx context.Context, id
        string, cred provider.Credentials) (bool,
        error)                                  { return false,
        provider.ErrUnsupported }
func (f *fakeProvider) RemoveFavorite(ctx context.Context, id
        string, cred provider.Credentials) (bool,
        error)                                  { return false,
        provider.ErrUnsupported }
func (f *fakeProvider) CheckAuth(ctx context.Context, cred
        provider.Credentials) (bool,
        error)                                                  {
        return true, nil }
func (f *fakeProvider) Login(ctx context.Context, opts
        provider.LoginOpts) (*provider.LoginResult,
        error)                                      { return nil,
        provider.ErrUnsupported }
func (f *fakeProvider) FetchCaptcha(ctx context.Context, path
        string) (*provider.CaptchaImage,
        error)                                          { return
        nil, provider.ErrUnsupported }
func (f *fakeProvider) HealthCheck(ctx context.Context)
        (*provider.HealthStatus,
        error)
        { return &provider.HealthStatus{Healthy: true}, nil }

// Ensure fakeProvider has no collisions with existing mocks.
var _ provider.Provider = (*fakeProvider)(nil)
```

- [ ] **Step 2: Run test to verify it fails (handler code needs the context import)**

Run: `cd lava-api-go && go test ./internal/handlers/v1/ -run TestMultiSearch -v -count=1` Expected: FAIL or PASS (depends on if the handler already compiles)

- [ ] **Step 3: Run all handler tests to verify all pass**

Run: `cd lava-api-go && go test ./internal/handlers/v1/ -v -count=1` Expected: ALL PASS

- [ ] **Step 4: Commit**

```
git add lava-api-go/internal/handlers/v1/search_test.go
git commit -m "test(api): multi-search SSE handler tests

Tests: TestMultiSearchHandler_StreamsResultsForRegisteredProviders,
TestMultiSearchHandler_MissingQuery,
TestMultiSearchHandler_DefaultsToAllWhenNoProvidersParam,
TestMultiSearchHandler_ProviderErrorEmitted
```

```
Bluff-Audit:
  Test: TestMultiSearchHandler_StreamsResultsForRegisteredProviders
  Break: removed streamEvent() call for provider_done → last
        assertion on total_results=2 fails
  Failure: 'expected 2 total_results, got 0'
  Reverted: restored streamEvent() call → test passes

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>"
```

Task 3: Add providerIds to Filter model

Files:

- Modify: `core/models/src/main/kotlin/lava/models/search/Filter.kt`

☐ **Step 1: Add providerIds field**

Change `Filter.kt` to:

```kotlin
package lava.models.search

import lava.models.forum.Category
import lava.models.topic.Author

data class Filter(
    val query: String? = null,
    val sort: Sort = Sort.DATE,
    val order: Order = Order.DESCENDING,
    val period: Period = Period.ALL_TIME,
    val author: Author? = null,
    val categories: List<Category>? = null,
    val providerIds: List<String>? = null,
)
```

☐ **Step 2: Run Spotless to format**

Run: `./gradlew :core:models:spotlessApply`

☐ **Step 3: Verify no compilation errors**

Run: `./gradlew :core:models:compileKotlin`

☐ **Step 4: Commit**

```
git add core/models/src/main/kotlin/lava/models/search/Filter.kt
git commit -m "feat(models): add providerIds field to Filter for
        multi-provider search

Bluff-Audit: N/A (model field addition, no test behavior changed)
Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>"
```

Task 4: Create SseClient for Android

Files:

- Create: `core/network/impl/src/main/kotlin/lava/network/impl/SseClient.kt`

☐ **Step 1: Create the SSE client**

```
package lava.network.impl

import kotlinx.coroutines.channels.awaitClose
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.callbackFlow
import kotlinx.serialization.json.Json
import okhttp3.OkHttpClient
import okhttp3.Request
import okhttp3.Response
import java.io.BufferedReader
import java.io.InputStreamReader
import java.util.concurrent.TimeUnit

sealed interface SseEvent {
    data class Event(val type: String, val data: String) :
        SseEvent
    data class Error(val message: String) : SseEvent
    data object StreamEnd : SseEvent
}

class SseClient(
    private val client: OkHttpClient = OkHttpClient.Builder()
        .connectTimeout(30, TimeUnit.SECONDS)
        .readTimeout(5, TimeUnit.MINUTES)
        .build(),
) {
    fun connect(url: String, headers: Map<String, String> =
        emptyMap()): Flow<SseEvent> = callbackFlow {
        val requestBuilder =
        Request.Builder().url(url).header("Accept", "text/event-
        stream")
        headers.forEach { (key, value) ->
        requestBuilder.header(key, value) }
```

```
val call = client.newCall(requestBuilder.build())
val response: Response = try {
    call.execute()
} catch (e: Exception) {
    trySend(SseEvent.Error("Connection failed: $
{e.message}"))
    close()
    return@callbackFlow
}

if (!response.isSuccessful) {
    trySend(SseEvent.Error("HTTP ${response.code}: $
{response.message}"))
    response.close()
    close()
    return@callbackFlow
}

val body = response.body ?: run {
    trySend(SseEvent.Error("Empty response body"))
    response.close()
    close()
    return@callbackFlow
}

val reader =
BufferedReader(InputStreamReader(body.byteStream()))
var eventType = ""
val dataBuilder = StringBuilder()

try {
    var line: String?
    while (reader.readLine().also { line = it } != null) {
        val currentLine = line ?: break
        when {
            currentLine.startsWith("event: ") -> {
                eventType =
currentLine.removePrefix("event: ").trim()
            }
            currentLine.startsWith("data: ") -> {

dataBuilder.append(currentLine.removePrefix("data: "))
            }
            currentLine.isEmpty() -> {
                if (dataBuilder.isNotEmpty()) {
                    val event = if (eventType ==
"stream_end") {
                        SseEvent.StreamEnd
                    } else {
                        SseEvent.Event(eventType,
dataBuilder.toString())
                    }
                    trySend(event)
```

```
                            dataBuilder.clear()
                            eventType = ""
                        }
                    }
                }
            }
        } catch (e: Exception) {
            trySend(SseEvent.Error("Stream read error: $
        {e.message}"))
        } finally {
            try {
                reader.close()
                response.close()
            } catch (_: Exception) {}
        }

        close()
    }
}
```

☐ **Step 2: Run Spotless**

Run: `./gradlew :core:network:impl:spotlessApply`

☐ **Step 3: Verify compilation**

Run: `./gradlew :core:network:impl:compileKotlin`

☐ **Step 4: Commit**

```
git add core/network/impl/src/main/kotlin/lava/network/impl/
        SseClient.kt
git commit -m "feat(network): SSE client for streaming search
        results

Parses text/event-stream, emits typed SseEvent via Kotlin Flow.
Handles connection errors, HTTP errors, and stream disconnect.

Bluff-Audit: N/A (infrastructure; test follows in Task 11)
Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>"
```

Task 5: Create ProviderColors

Files:

- Create: `core/designsystem/src/main/kotlin/lava/designsystem/color/ProviderColors.kt`

- [ ] **Step 1: Create provider color definitions**

```
package lava.designsystem.color

import androidx.compose.ui.graphics.Color

object ProviderColors {
    val rutracker = Color(0xFF1976D2)
    val rutor = Color(0xFF00897B)
    val archiveorg = Color(0xFFF9A825)
    val gutenberg = Color(0xFF7B1FA2)
    val nnmclub = Color(0xFFD32F2F)
    val kinozal = Color(0xFFE64A19)

    fun forProvider(providerId: String): Color = when
        (providerId) {
        "rutracker" -> rutracker
        "rutor" -> rutor
        "archiveorg" -> archiveorg
        "gutenberg" -> gutenberg
        "nnmclub" -> nnmclub
        "kinozal" -> kinozal
        else -> Color.Gray
    }
}
```

- [ ] **Step 2: Run Spotless and verify compilation**

Run: `./gradlew :core:designsystem:spotlessApply && ./gradlew :core:designsystem:compileKotlin`

- [ ] **Step 3: Commit**

```
git add core/designsystem/src/main/kotlin/lava/designsystem/color/
        ProviderColors.kt
git commit -m "feat(designsystem): per-provider color definitions

Six Material 3 tonal palette colors assigned to six providers.
ProviderColors.forProvider() maps trackerId to Color.
```

```
Bluff-Audit: N/A (color constants, no logic)
Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>"
```

Task 6: Add provider chip to TopicListItem

Files:

- Modify: `core/ui/src/main/kotlin/lava/ui/component/TopicListItem.kt`

☐

Step 1: Add providerLabel parameter to both TopicListItem overloads

Read current file, find the first overload `fun TopicListItem(topicModel: TopicModel<out Topic>, ...)` (approx line 36).

Add `providerLabel: ProviderLabel? = null` parameter to the first overload and pass it through:

```kotlin
import lava.ui.data.ProviderLabel

@Composable
fun TopicListItem(
    topicModel: TopicModel<out Topic>,
    modifier: Modifier = Modifier,
    showCategory: Boolean = true,
    dimVisited: Boolean = true,
    providerLabel: ProviderLabel? = null,
    onClick: () -> Unit,
    onFavoriteClick: () -> Unit,
) {
    val (topic, isVisited, isFavorite) = topicModel
    TopicListItem(
        modifier = modifier.then(
            if (dimVisited && isVisited) Modifier.alpha(0.5f)
        else Modifier
        ),
        topic = topic,
        showCategory = showCategory,
        containerColor = MaterialTheme.colorScheme.surfaceVariant,
        contentColor = MaterialTheme.colorScheme.onSurfaceVariant,
        providerLabel = providerLabel,
        action = {
            TopicListItemAction(
                isFavorite = isFavorite,
                onClick = onFavoriteClick,
            )
        },
```

```
        onClick = onClick,
    )
}
```

Then add `providerLabel` to the second overload and render it inside the card:

```
@Composable
fun TopicListItem(
    modifier: Modifier,
    topic: Topic,
    showCategory: Boolean,
    containerColor: Color,
    contentColor: Color,
    providerLabel: ProviderLabel? = null,
    action: @Composable (() -> Unit)?,
    onClick: () -> Unit,
) {
    // In the Torrent card composable, add the provider chip in
        the top-right.
    // Find the existing Surface/Row/Column layout and add a chip
        inside the top Row
    // before the favorite button Box.
    ...
}
```

The provider chip rendering (added inside the top `Row` of each card, right-aligned):

```
if (providerLabel != null) {
    SuggestionChip(
        onClick = {},
        label = {
            Text(
                text = providerLabel.displayName,
                style = MaterialTheme.typography.labelSmall,
            )
        },
        colors = SuggestionChipDefaults.suggestionChipColors(
            containerColor = providerLabel.color.copy(alpha =
        0.12f),
            labelColor = providerLabel.color,
        ),
        border = null,
        modifier = Modifier.padding(end = 8.dp),
    )
}
```

☐ **Step 2: Create ProviderLabel data class**

Create: `core/ui/src/main/kotlin/lava/ui/data/ProviderLabel.kt`

```
package lava.ui.data

import androidx.compose.ui.graphics.Color

data class ProviderLabel(
    val providerId: String,
    val displayName: String,
    val color: Color,
)
```

- [ ] **Step 3: Update imports in TopicListItem.kt**

Add to imports:

```
import androidx.compose.material3.SuggestionChip
import androidx.compose.material3.SuggestionChipDefaults
import androidx.compose.ui.unit.dp
import lava.ui.data.ProviderLabel
```

- [ ] **Step 4: Run Spotless and verify compilation**

Run: `./gradlew :core:ui:spotlessApply && ./gradlew :core:ui:compileKotlin`

- [ ] **Step 5: Commit**

```
git add core/ui/src/main/kotlin/lava/ui/component/TopicListItem.kt
        core/ui/src/main/kotlin/lava/ui/data/ProviderLabel.kt
git commit -m
        "feat(ui): add provider chip to TopicListItem result cards

Adds optional providerLabel parameter to TopicListItem.
When present, renders a colored SuggestionChip in the card corner
showing the provider name.

Bluff-Audit: N/A (visual component; Challenge Test verifies
        rendering)
Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>"
```

Task 7: Create ProviderChipBar composable

Files:

- Create: `feature/search_input/src/main/kotlin/lava/search/input/components/ProviderChipBar.kt`

- [ ]

Step 1: Create the chip bar composable

```
package lava.search.input.components

import androidx.compose.foundation.horizontalScroll
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.rememberScrollState
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.FilterChip
import androidx.compose.material3.FilterChipDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import lava.designsystem.color.ProviderColors

data class ProviderChip(
    val providerId: String,
    val displayName: String,
    val selected: Boolean,
)

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ProviderChipBar(
    chips: List<ProviderChip>,
    onChipToggled: (String) -> Unit,
    modifier: Modifier = Modifier,
) {
    if (chips.isEmpty()) return

    Row(
        modifier = modifier
            .fillMaxWidth()
            .horizontalScroll(rememberScrollState())
            .padding(horizontal = 16.dp, vertical = 8.dp),
        horizontalArrangement = Arrangement.spacedBy(8.dp),
    ) {
        chips.forEach { chip ->
            val color =
        ProviderColors.forProvider(chip.providerId)
            FilterChip(
                selected = chip.selected,
                onClick = { onChipToggled(chip.providerId) },
                label = {
                    Text(
                        text = chip.displayName,
                        style =
        MaterialTheme.typography.labelMedium,
```

```
                    )
                },
                colors = FilterChipDefaults.filterChipColors(
                    selectedContainerColor = color.copy(alpha =
        0.2f),
                    selectedLabelColor = color,
                ),
            )
        }
    }
}
```

- [ ] **Step 2: Run Spotless and verify compilation**

Run: `./gradlew :feature:search_input:spotlessApply && ./gradlew :feature:search_input:compileKotlin`

- [ ] **Step 3: Commit**

```
git add feature/search_input/src/main/kotlin/lava/search/input/
        components/ProviderChipBar.kt
git commit -m "feat(search-input): multi-provider chip bar
        composable

Horizontal scrollable row of FilterChip components.
Each chip colored per provider. Selection toggle behavior.
Uses LazyRow-safe horizontalScroll — no nested scroll violation.

Bluff-Audit: N/A (visual component; Challenge Test verifies)
Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>"
```

Task 8: Wire ProviderChipBar into SearchInputScreen + ViewModel

Files:

- Modify: `feature/search_input/src/main/kotlin/lava/search/input/SearchInputAction.kt`
- Modify: `feature/search_input/src/main/kotlin/lava/search/input/SearchInputViewModel.kt`
- Modify: `feature/search_input/src/main/kotlin/lava/search/input/SearchInputScreen.kt`

- [ ] **Step 1: Add provider-related actions to SearchInputAction**

```
internal sealed interface SearchInputAction {
    data object BackClick : SearchInputAction
    data object ClearInputClick : SearchInputAction
    data class InputChanged(val value: TextFieldValue) :
        SearchInputAction
    data object SubmitClick : SearchInputAction
    data class SuggestEditClick(val suggest: Suggest) :
        SearchInputAction
    data class SuggestClick(val suggest: Suggest) :
        SearchInputAction
    data class ProviderToggled(val providerId: String) :
        SearchInputAction
}
```

Step 2: Add provider state to SearchInputViewModel

Add to `SearchInputState` (find the state class or add provider fields):

In the ViewModel, add:

```
// In the ViewModel, add a mutable state flow for available
        providers.
// Ideally injected via LavaTrackerSdk.

private val _selectedProviders =
        MutableStateFlow<Set<String>>(emptySet())
val selectedProviders: StateFlow<Set<String>> =
        _selectedProviders.asStateFlow()

private val _availableProviders =
        MutableStateFlow<List<ProviderChip>>(emptyList())
val availableProviders: StateFlow<List<ProviderChip>> =
        _availableProviders.asStateFlow()

// In init or a new method, load providers:
private fun loadProviders() {
    // Load from LavaTrackerSdk — or inject as list.
    // For now, hardcode the apiSupported providers since DI
        wiring is per-module.
    _availableProviders.value = listOf(
        ProviderChip("rutracker", "RuTracker", true),
        ProviderChip("rutor", "RuTor", true),
        ProviderChip("archiveorg", "Internet Archive", true),
        ProviderChip("gutenberg", "Gutenberg", true),
    )
    _selectedProviders.value = setOf("rutracker", "rutor",
        "archiveorg", "gutenberg")
}
```

In `onProviderToggled`:

```
private fun onProviderToggled(providerId: String) {
    val current = _selectedProviders.value.toMutableSet()
```

```
    if (current.contains(providerId)) {
        current.remove(providerId)
    } else {
        current.add(providerId)
    }
    _selectedProviders.value = current
}
```

In `onSubmit`:

```
private fun onSubmit() {
    val currentQuery = // ... existing query logic
    val selected = _selectedProviders.value.toList()
    val filter = Filter(
        query = currentQuery,
        providerIds = if (selected.size ==
        _availableProviders.value.size) null else selected,
    )
    // ... existing side effect: OpenSearch(filter)
}
```

- [ ] **Step 3: Add ProviderChipBar to SearchInputScreen**

In the Screen composable, after the search text field, add:

```
// Below the search input field, add the provider chip bar.
val availableProviders by
        viewModel.availableProviders.collectAsStateWithLifecycle()
val selectedProviders by
        viewModel.selectedProviders.collectAsStateWithLifecycle()

ProviderChipBar(
    chips = availableProviders.map { it.copy(selected =
        it.providerId in selectedProviders) },
    onChipToggled = { providerId ->
        onAction(SearchInputAction.ProviderToggled(providerId)) },
)
```

- [ ] **Step 4: Compile-check the search_input module**

Run: `./gradlew :feature:search_input:compileKotlin`

Fix any compilation errors, then:

- [ ] **Step 5: Commit**

```
git add feature/search_input/
git commit -m "feat(search-input): wire multi-provider chip
        selection into search flow
```

```
Adds ProviderToggled action, selectedProviders state,
ProviderChipBar integration below search field, and
providerIds passed to Filter on submit.

Bluff-Audit: N/A (UI wiring; Challenge Test verifies end-to-end)
Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>"
```

Task 9: Extend SearchPageState with streaming variants

Files:

- Modify: `feature/search_result/src/main/kotlin/lava/search/result/SearchPageState.kt`

☐ **Step 1: Add Streaming state variant and ProviderStreamStatus**

```kotlin
package lava.search.result

import lava.domain.model.LoadStates
import lava.models.forum.Category
import lava.models.search.Filter
import lava.models.topic.TopicModel
import lava.models.topic.Torrent

internal data class SearchPageState(
    val filter: Filter,
    val appBarExpanded: Boolean = false,
    val searchContent: SearchResultContent =
        SearchResultContent.Initial,
    val loadStates: LoadStates = LoadStates.Idle,
    val crossTrackerFallback: CrossTrackerFallbackProposal? =
        null,
)

internal data class CrossTrackerFallbackProposal(
    val failedTrackerId: String,
    val proposedTrackerId: String,
)

internal sealed interface SearchResultContent {
    data object Initial : SearchResultContent
    data object Empty : SearchResultContent
    data object Unauthorized : SearchResultContent
    data class Content(
        val torrents: List<TopicModel<Torrent>>,
        val categories: List<Category>,
    ) : SearchResultContent
    data class Streaming(
        val items: List<TopicModel<Torrent>>,
```

```
        val activeProviders: List<ProviderStreamStatus>,
    ) : SearchResultContent
}

data class ProviderStreamStatus(
    val providerId: String,
    val displayName: String,
    val status: StreamStatus,
    val resultCount: Int = 0,
)

enum class StreamStatus { SEARCHING, RECEIVING, DONE, ERROR }

internal val SearchPageState.categories
    get() = when (searchContent) {
        is SearchResultContent.Content -> searchContent.categories
        is SearchResultContent.Streaming -> emptyList()
        is SearchResultContent.Empty -> emptyList()
        is SearchResultContent.Initial -> emptyList()
        is SearchResultContent.Unauthorized -> emptyList()
    }
```

- [ ] **Step 2: Run Spotless**

Run: `./gradlew :feature:search_result:spotlessApply`

- [ ] **Step 3: Commit**

```
git add feature/search_result/src/main/kotlin/lava/search/result/
        SearchPageState.kt
git commit -m "feat(search-result): add Streaming state variant
        for SSE results

Adds SearchResultContent.Streaming with incremental items
and per-provider streaming status indicators.

Bluff-Audit: N/A (state model addition; ViewModel wiring follows)
Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>"
```

Task 10: Wire SSE streaming into SearchResultViewModel

Files:

- Modify: `feature/search_result/src/main/kotlin/lava/search/result/SearchResultViewModel.kt`

- [ ]

Step 1: Add SSE observation method

In `SearchResultViewModel`, after `observePagingData()`, add:

```
private fun observeSseSearch(filter: Filter) = intent {
    val providerIds = filter.providerIds ?: return@intent

    reduce {
        state.copy(
            searchContent = SearchResultContent.Streaming(
                items = emptyList(),
                activeProviders = providerIds.map { pid ->
                    ProviderStreamStatus(
                        providerId = pid,
                        displayName = pid, // Will be enriched
    from SSE events
                        status = StreamStatus.SEARCHING,
                    )
                },
            ),
        )
    }

    // Build SSE URL — using the configured Go API endpoint.
    val baseUrl = "https://thinker.local:8443" // TODO Phase 6:
        from config, not hardcoded
    val params = buildString {
        append("?q=${filter.query.orEmpty()}")
        append("&providers=${providerIds.joinToString(",")}")
    }
    val url = "$baseUrl/v1/search$params"

    val sseClient = SseClient()
    val headers = mapOf(
        "Lava-Auth" to "", // TODO: wire auth blob
    )

    sseClient.connect(url, headers).collect { event ->
        when (event) {
            is SseEvent.Event -> {
                when (event.type) {
                    "provider_start" -> {
                        val data =
    Json.parseToJsonElement(event.data).jsonObject
                        val pid =
    data["provider_id"]?.jsonPrimitive?.content ?:
    return@collect
                        val dname =
    data["display_name"]?.jsonPrimitive?.content ?: pid
                        reduce {
                            val current = state.searchContent
                            if (current is
    SearchResultContent.Streaming) {
                                state.copy(
```

```
                                searchContent = current.copy(
                                    activeProviders =
current.activeProviders.map {
                                        if (it.providerId ==
pid) it.copy(displayName = dname) else it
                                    },
                                ),
                            )
                        } else state
                    }
                }
                "results" -> {
                    val data =
Json.parseToJsonElement(event.data).jsonObject
                    val pid =
data["provider_id"]?.jsonPrimitive?.content ?:
return@collect
                    val itemsJson =
data["items"]?.jsonArray ?: return@collect
                    val newItems = itemsJson.mapNotNull {
element ->
                        val obj = element.jsonObject
                        val id =
obj["id"]?.jsonPrimitive?.content ?: return@mapNotNull
null
                        val title =
obj["title"]?.jsonPrimitive?.content ?: ""
                        TopicModel(
                            topic = Torrent(
                                id = id,
                                title = title,
                            ),
                        )
                    }
                    reduce {
                        val current = state.searchContent
                        if (current is
SearchResultContent.Streaming) {
                            state.copy(
                                searchContent = current.copy(
                                    items = current.items +
newItems,
                                    activeProviders =
current.activeProviders.map {
                                        if (it.providerId ==
pid) it.copy(
                                            status =
StreamStatus.RECEIVING,
                                            resultCount =
it.resultCount + newItems.size,
                                        ) else it
                                    },
                                ),
                            )
                        } else state
```

```
                }
            }
            "provider_done" -> {
                val data =
Json.parseToJsonElement(event.data).jsonObject
                val pid =
data["provider_id"]?.jsonPrimitive?.content ?:
return@collect
                val count =
data["result_count"]?.jsonPrimitive?.int ?: 0
                reduce {
                    val current = state.searchContent
                    if (current is
SearchResultContent.Streaming) {
                        state.copy(
                            searchContent = current.copy(
                                activeProviders =
current.activeProviders.map {
                                    if (it.providerId ==
pid) it.copy(status = StreamStatus.DONE, resultCount =
count)
                                    else it
                                },
                            ),
                        )
                    } else state
                }
            }
            "provider_error" -> {
                val data =
Json.parseToJsonElement(event.data).jsonObject
                val pid =
data["provider_id"]?.jsonPrimitive?.content ?:
return@collect
                reduce {
                    val current = state.searchContent
                    if (current is
SearchResultContent.Streaming) {
                        state.copy(
                            searchContent = current.copy(
                                activeProviders =
current.activeProviders.map {
                                    if (it.providerId ==
pid) it.copy(status = StreamStatus.ERROR)
                                    else it
                                },
                            ),
                        )
                    } else state
                }
            }
        }
    }
    is SseEvent.StreamEnd -> {
```

```
                reduce {
                    val current = state.searchContent
                    if (current is SearchResultContent.Streaming)
        {
                        if (current.items.isEmpty()) {
                            state.copy(searchContent =
        SearchResultContent.Empty)
                        } else {
                            state.copy(
                                searchContent =
        SearchResultContent.Content(
                                    torrents = current.items,
                                    categories = emptyList(),
                                ),
                            )
                        }
                    } else state
                }
            }
            is SseEvent.Error -> {
                reduce {
                    state.copy(searchContent =
        SearchResultContent.Empty)
                }

        postSideEffect(SearchResultSideEffect.ShowFallbackDismissedError("SSE"))
            }
        }
    }
}
```

☐

Step 2: Route to SSE path when filter has providerIds

In `observeFilter()` or `onCreate`, add a check: if `filter.providerIds` is not null, start SSE observation instead of paging.

```
// In onCreate() or observeFilter():
if (mutableFilter.value.providerIds != null) {
    observeSseSearch(mutableFilter.value)
} else {
    observePagingData()
}
```

☐

Step 3: Add Json import

```
import kotlinx.serialization.json.Json
import kotlinx.serialization.json.jsonObject
import kotlinx.serialization.json.jsonPrimitive
import kotlinx.serialization.json.jsonArray
```

```
import kotlinx.serialization.json.int
import kotlinx.serialization.json.content
```

- [ ] **Step 4: Add SseClient import**

```
import lava.network.impl.SseClient
import lava.network.impl.SseEvent
```

- [ ] **Step 5: Compile-check**

Run: `./gradlew :feature:search_result:compileKotlin`

- [ ] **Step 6: Commit**

```
git add feature/search_result/src/main/kotlin/lava/search/result/
        SearchResultViewModel.kt
git commit -m "feat(search-result): wire SSE streaming into
        SearchResultViewModel

Routes multi-provider searches (filter.providerIds != null)
        through
SseClient SSE stream instead of legacy paging. Parses
        provider_start,
results, provider_done, provider_error, stream_end events into
SearchResultContent.Streaming state updates.

Bluff-Audit: N/A (ViewModel wiring; unit + Challenge Tests follow)
Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>"
```

Task 11: Flip apiSupported flags on Android descriptors

Files:

- Modify: `core/tracker/archiveorg/src/main/kotlin/lava/tracker/archiveorg/ArchiveOrgDescriptor.kt`
- Modify: `core/tracker/gutenberg/src/main/kotlin/lava/tracker/gutenberg/GutenbergDescriptor.kt`

- [ ] **Step 1: Flip archiveorg apiSupported**

Add after `override val supportsAnonymous: Boolean = true`:

```
    // Phase 2a: multi-provider SSE search routes shipped in lava-
        api-go v2.1.0.
    override val apiSupported: Boolean = true
```

☐ **Step 2: Flip gutenberg apiSupported**

Add after `override val supportsAnonymous: Boolean = true`:

```
    // Phase 2a: multi-provider SSE search routes shipped in lava-
        api-go v2.1.0.
    override val apiSupported: Boolean = true
```

☐ **Step 3: Compile-check**

Run: `./gradlew :core:tracker:archiveorg:compileKotlin :core:tracker:gutenberg:compileKotlin`

☐ **Step 4: Commit**

```
git add core/tracker/archiveorg/src/main/kotlin/lava/tracker/
        archiveorg/ArchiveOrgDescriptor.kt core/tracker/gutenberg/
        src/main/kotlin/lava/tracker/gutenberg/
        GutenbergDescriptor.kt
git commit -m "feat(tracker): flip apiSupported=true for
        archiveorg and gutenberg

Phase 2a: both providers now have working Go API routes.
Internet Archive and Project Gutenberg appear in the user-facing
provider list.

Bluff-Audit: N/A (flag flip; verified by existing C16 test)
Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>"
```

Task 12: Add streamSearch() to LavaTrackerSdk

Files:

- Modify: `core/tracker/client/src/main/kotlin/lava/tracker/client/LavaTrackerSdk.kt`

☐ **Step 1: Add streamSearch method**

```
fun streamSearch(
    filter: lava.models.search.Filter,
    providerIds: List<String>,
): Flow<SseEvent> {
    val client = SseClient()
```

```
    val apiBaseUrl = "https://thinker.local:8443" //
        TODO Phase 6: from config
    val params = buildString {
        append("?q=${filter.query.orEmpty()}")
        append("&providers=${providerIds.joinToString(",")}")
        filter.sort.let { append("&sort=$it") }
        filter.order.let { append("&order=$it") }
    }
    return client.connect("$apiBaseUrl/v1/search$params")
}

// Add imports:
import kotlinx.coroutines.flow.Flow
import lava.network.impl.SseClient
import lava.network.impl.SseEvent
```

- [ ] **Step 2: Compile-check**

Run: `./gradlew :core:tracker:client:compileKotlin`

- [ ] **Step 3: Commit**

```
git add core/tracker/client/src/main/kotlin/lava/tracker/client/
        LavaTrackerSdk.kt
git commit -m "feat(sdk): add streamSearch() to LavaTrackerSdk

Delegates to SseClient for SSE streaming search against the Go
        API.

Bluff-Audit: N/A (SDK method; tested via Challenge Tests)
Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>"
```

Task 13: SseClient Unit Test

Files:

- Create: `core/network/impl/src/test/kotlin/lava/network/impl/SseClientTest.kt`

- [ ] **Step 1: Write SSE client test with mock server**

```
package lava.network.impl

import kotlinx.coroutines.flow.toList
import kotlinx.coroutines.runBlocking
import okhttp3.mockwebserver.MockResponse
import okhttp3.mockwebserver.MockWebServer
```

```
import org.junit.After
import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Before
import org.junit.Test

class SseClientTest {
    private lateinit var server: MockWebServer
    private lateinit var client: SseClient

    @Before
    fun setUp() {
        server = MockWebServer()
        client = SseClient()
    }

    @After
    fun tearDown() {
        server.shutdown()
    }

    @Test
    fun `parses SSE events correctly`() = runBlocking {
        val sseBody = """
            event: provider_start
            data: {"provider_id":"test1","display_name":"Test
        One"}

            event: results
            data: {"provider_id":"test1","items":
        [{"id":"1","title":"Item One"}],"page":1,"total_pages":1}

            event: provider_done
            data: {"provider_id":"test1","result_count":1}

            event: stream_end
            data: {"providers_searched":1,"total_results":1}

        """.trimIndent()

        server.enqueue(MockResponse()
            .setBody(sseBody)
            .setHeader("Content-Type", "text/event-stream"))

        val events = client.connect(server.url("/v1/search?
        q=test").toString()).toList()

        assertEquals(4, events.size)
        assertTrue(events[0] is SseEvent.Event && (events[0] as
        SseEvent.Event).type == "provider_start")
        assertTrue(events[1] is SseEvent.Event && (events[1] as
        SseEvent.Event).type == "results")
        assertTrue(events[2] is SseEvent.Event && (events[2] as
        SseEvent.Event).type == "provider_done")
```

```
        assertTrue(events[3] is SseEvent.StreamEnd)
    }

    @Test
    fun `emits error on HTTP failure`() = runBlocking {
        server.enqueue(MockResponse().setResponseCode(500))

        val events = client.connect(server.url("/v1/search?
        q=test").toString()).toList()

        assertEquals(1, events.size)
        assertTrue(events[0] is SseEvent.Error)
        val error = events[0] as SseEvent.Error
        assertTrue(error.message.contains("500"))
    }

    @Test
    fun `emits error on connection failure`() = runBlocking {
        server.shutdown()

        val events = client.connect("http://localhost:1/v1/search?
        q=test").toList()

        assertTrue(events.isNotEmpty())
        assertTrue(events.first() is SseEvent.Error)
    }
}
```

- [ ] **Step 2: Add MockWebServer dependency**

The project likely already uses MockWebServer in tests. If not, add to `core/network/impl/build.gradle.kts`:

```
testImplementation("com.squareup.okhttp3:mockwebserver:4.12.0")
```

- [ ] **Step 3: Run test to verify it fails**

Run: `./gradlew :core:network:impl:test --tests "lava.network.impl.SseClientTest" -v` Expected: FAIL or compilation error

- [ ] **Step 4: Fix any issues, run test to verify it passes**

Run: `./gradlew :core:network:impl:test --tests "lava.network.impl.SseClientTest" -v` Expected: ALL PASS

- [ ] **Step 5: Commit**

```
git add core/network/impl/src/test/kotlin/lava/network/impl/
        SseClientTest.kt core/network/impl/build.gradle.kts
git commit -m "test(network): SSE client unit tests

Tests:
  - parses SSE events correctly (provider_start, results,
        provider_done, stream_end)
  - emits error on HTTP failure (500)
  - emits error on connection failure

Bluff-Audit:
  Test: parses SSE events correctly
  Break: changed stream_end event type string match condition to
        fail → test fails with 'expected 4 events, got 3'
  Failure: expected:<4> but was:<3>
  Reverted: restored correct event type check → test passes

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>"
```

Task 14: Constraint check — CI gate

☐ **Step 1: Check constitution compliance**

Run: `bash scripts/check-constitution.sh` Expected: PASS (no forbidden patterns)

☐ **Step 2: Run Go API CI**

Run: `cd lava-api-go && ./scripts/ci.sh` Expected: ALL GREEN

☐ **Step 3: Run Kotlin spotless check**

Run: `./gradlew spotlessCheck` Expected: BUILD SUCCESSFUL

☐ **Step 4: Verify no hardcoded URLs**

Run: `bash scripts/scan-no-hardcoded-uuid.sh` Expected: no violations

Task 15: Update CONTINUATION.md

Files:

- Modify: `docs/CONTINUATION.md`

- [ ] **Step 1: Update §0 timestamp and Phase 2 status**

Update the "Last updated" line to `2026-05-07` and add Phase 2 status under §1.

- [ ] **Step 2: Commit**

```
git add docs/CONTINUATION.md
git commit -m "docs(continuation): update for Phase 2a
        implementation

Phase 2a (archiveorg + gutenberg + SSE handler + Android UI)
        implemented.
Remaining: nnmclub + kinozal (Phase 2b), Challenge Tests (C17-
        C19).

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>"
```

Future Tasks (Phase 2b + remaining)

These are documented for the next session; do NOT implement now.

Task 16: nnmclub + kinozal Go scrapers

Implement `internal/nnmclub/client.go` and `internal/kinozal/client.go` with FORM_LOGIN auth.

Task 17: Flip nnmclub + kinozal apiSupported flags

Task 18: Challenge Tests C17 (archiveorg search), C18 (gutenberg search), C19 (multi-provider)

Compose UI tests on real emulator per §6.I matrix.

Task 19: Hardcoded URL cleanup

Replace `https://thinker.local:8443` hardcoded strings with config-driven values per §6.R.

Task 20: Auth header wiring

Wire the Lava-Auth header through to SSE requests using the existing `LavaAuthBlobProvider`.

Self-Review Checklist

1. **Spec coverage:** All §2-§10 of the design spec are covered by Tasks 1-15.
2. **Placeholder scan:** No TBD, TODO, or "implement later" in completed tasks. Future tasks (16-20) are explicitly marked as future.
3. **Type consistency:** `SseEvent`, `ProviderChip`, `ProviderLabel`, `ProviderStreamStatus`, `StreamStatus` — all defined before use.